

DM2026 • LAB 1

The Agentic Exercise

Teaching an AI agent to build, and then run, your own data-mining tool pipeline.

THE BUILDING BLOCKS

LLM + tools + harness = agent

LLM

Just the model

Gemini or Groq. Predicts the next token from whatever text it's given. No memory between calls, can't read a file or run code on its own.

Tool

A typed function

A Python function with a name, typed parameters, and a docstring. The model can request it by name; it never runs it itself.

Harness

The loop around it

Resends the conversation every call, notices a tool request, runs it, feeds the result back in.

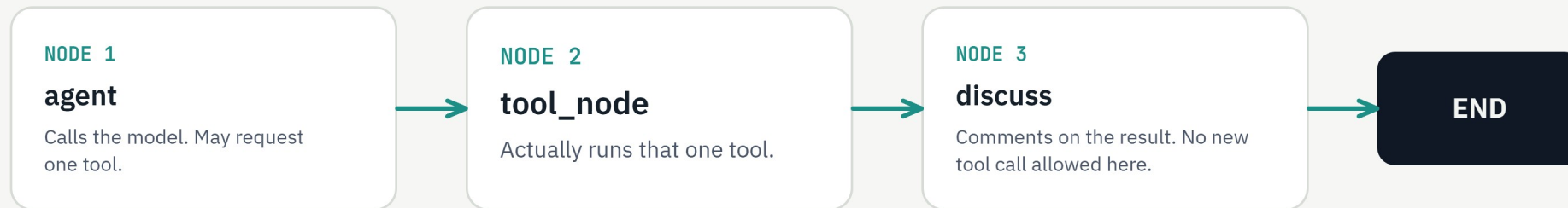
Put together: an agent

An LLM plus tools plus a harness. Together they let the model's predicted text turn into a real action, like writing a file, instead of staying just words on a screen.

Want more detail? Anthropic's [Building Effective AI Agents](#) and [Effective Harnesses for Long-Running Agents](#).

LangGraph: named steps, one call at a time

graph_core.py builds the harness as a graph: named steps ("nodes") connected by edges that say what runs next. Three nodes, always in this order:



No tool call that turn? The agent node skips straight to END -- no tool_node, no discuss step. One tool call, then a mandatory comment, every single time, never the other way around.

Same pattern powers real coding agents

Claude Code and OpenAI's Codex run this same loop: an LLM proposes an action, a harness runs it and reports back, you direct the next step. Steering this lab's harness well, specific requests, reading a proposal before approving, testing before trusting a result, is the same skill professional engineers use directing those tools.

Want more detail? docs.langchain.com/oss/python/langgraph/overview, or the free official course at academy.langchain.com/courses/langgraph-essentials-python.

WHY THIS MATTERS

Agentic AI makes you faster, not lazier

Data mining has always meant spending real time on two very different things: typing correct code, and deciding what the data actually means. Agentic AI moves the balance of your time toward the second one.

Less time on syntax

CountVectorizer calls, PAMI's transactional-database format, matplotlib boilerplate: the agent handles the API details. Your job is the threshold, the method, the category to test.

Cheap to re-test

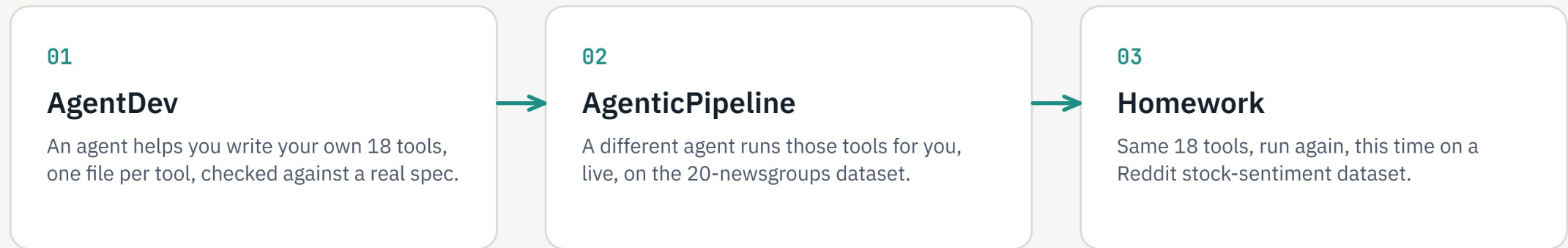
Homework runs your exact pipeline on a different dataset in one session. Hand-coding that same generalization check would usually cost more time than most people are willing to spend.

More room to interpret

Time saved on typing goes into asking why a term survived a filter, why a pattern's support dropped, whether a result actually makes sense.

Faster is not the same as easier. The grading in this lab rewards specific direction and real engagement with results, exactly the habits that make an agent actually save you time instead of just moving the typing to a different place.

Three notebooks, one pipeline



Builds on the data-mining concepts taught in **DM2026-Lab1-Master.ipynb**, do that one first.

AgenticPipeline and Homework both run the exact tools built in AgentDev, on two different datasets, to check whether what you built actually generalizes.

One glossary, every notebook

39 terms, all in **agent_dev/GLOSSARY.md**, split into three parts. Every notebook and reference page in the course points back to the same page.

18

AI agent terms

LLM, tool, harness, backend, rate limit, and more.

16

Programming terms

Function, type hint, dict, JSON, and more.

5

Testing terms

pytest, assert, known answer, and more.

How the link works

The first time a notebook uses a term, it's **bold** with a small number right after it, for example tools (glossary #5). That number is the entry to jump to on this page.

Three files, one source of truth

Before the agent proposes code, a test, or a parameter type, it has to read these three files. None of it is pre-baked into its instructions.

TOOLS_PIPELINE_BREAKDOWN.md

The contract for all 20 tools: what each one takes, returns, and stores.

TEST_FIXTURE.md

21 known answers on one tiny 8-row dataset, so every tool has a real number to test against.

BACKEND_SCHEMA_NOTES.md

Which parameter types silently break on one LLM backend but work fine on the other.

read_spec_file_tool

A narrow, hardcoded tool that can open exactly these three files and nothing else. No reference implementation, no path outside the workspace it can wander into.

1 · agent_dev/

2 · agent_dev_workspace/

3 • plots/

4 • session logs/

5 · reports/

6 • The three notebooks

IMPORTANT: Always use the same name and student ID across your notebooks. If they don't match, your grading may be affected.

Also: `agent_dev_workspace/`, `session_logs/`, `plots/`, and `reports/` don't exist yet when you fork this repo. They're created automatically once you actually run the notebooks, not something you need to make yourself.

Running your tools for real

0

Setup

Your name and student ID, a backend (Groq or Gemini), and its API key.

1

Discover tools

The 18 you built in AgentDev, plus `load_dataset_tool` and `visualize_result_tool`, already provided.

2

Direct the analysis

One tool call per turn, on 20-newsgroups: `alt.atheism`, `soc.religion.christian`, `comp.graphics`, `sci.med`.

3

Everything is logged

The full conversation is encrypted as you go. That log file is what you submit.

Same tools, new data

Nothing about your tools should need to change. If one only works on 20-newsgroups, that's the bug this notebook is built to catch.

	AgenticPipeline	Homework
Dataset	20-newsgroups, via scikit-learn	newdataset/Reddit-stock-sentiment.csv
Tools used	Your 18, plus the 2 provided	The exact same 18, unchanged
Point of it	Show the pipeline works	Show it actually generalizes

Two things get looked at, not your code

YOUR SESSION LOG

Directing the agent

AgentDev: spec-driven requests naming the contract and known answers, reviewing every diff, actually testing what you build.

AgenticPipeline & Homework: which tools you call, in what order, how specifically, engagement with what came back.

YOUR REPORT

Written report

Coverage, accuracy against your own results, specific references, clear enough to stand alone, real interpretation over restatement.

AgenticPipeline and Homework only; AgentDev has no report at all.

`reports/Name_StudentID_track_report.md`, its own file,
written there directly, not inside the notebook

`session_logs/`, `plots/`, and `reports/` are all created for you automatically the first time you run each notebook's setup cells, nothing to create by hand. See slide 8 for where each one lives in the directory.

Save your progress the smart way

AgentDev: safe to pause

Each tool is one independent file, built and tested against the fixture on its own.

Resume any day: point the agent at the next tool by name. Nothing to redo.

AgenticPipeline & Homework: costly to pause

A new kernel session means a blank state: the loaded dataset, the document-term matrix, every stored result, all gone.

Resuming means redoing earlier steps first, not continuing where you left off.

Our advice

Start AgenticPipeline and Homework on a fresh rate limit, not a nearly-exhausted one. Running out of quota mid-pipeline forces that same restart-from-scratch cost, worse, mid-analysis instead of at a natural stopping point.

RECAP

An LLM, tools, and a harness you learn to direct

LLM + tools + harness = agent. This lab's harness is LangGraph: one tool call, then a mandatory comment, every time, the same pattern real coding agents like Claude Code and Codex run on.

AgentDev builds the eighteen tools; **AgenticPipeline** and **Homework** run those same tools on two different datasets, to check they generalize.

The **glossary** gives every term one home; the three **spec files** give every tool one source of truth.

Graded on how you direct the agent, all three notebooks, and, on the two run-notebooks, your written report; AgentDev is safe to pause, the two run-notebooks are not, so start those on a fresh rate limit.

Read the spec. Ask for one specific thing at a time. Let the agent build it and test it, not just generate it.